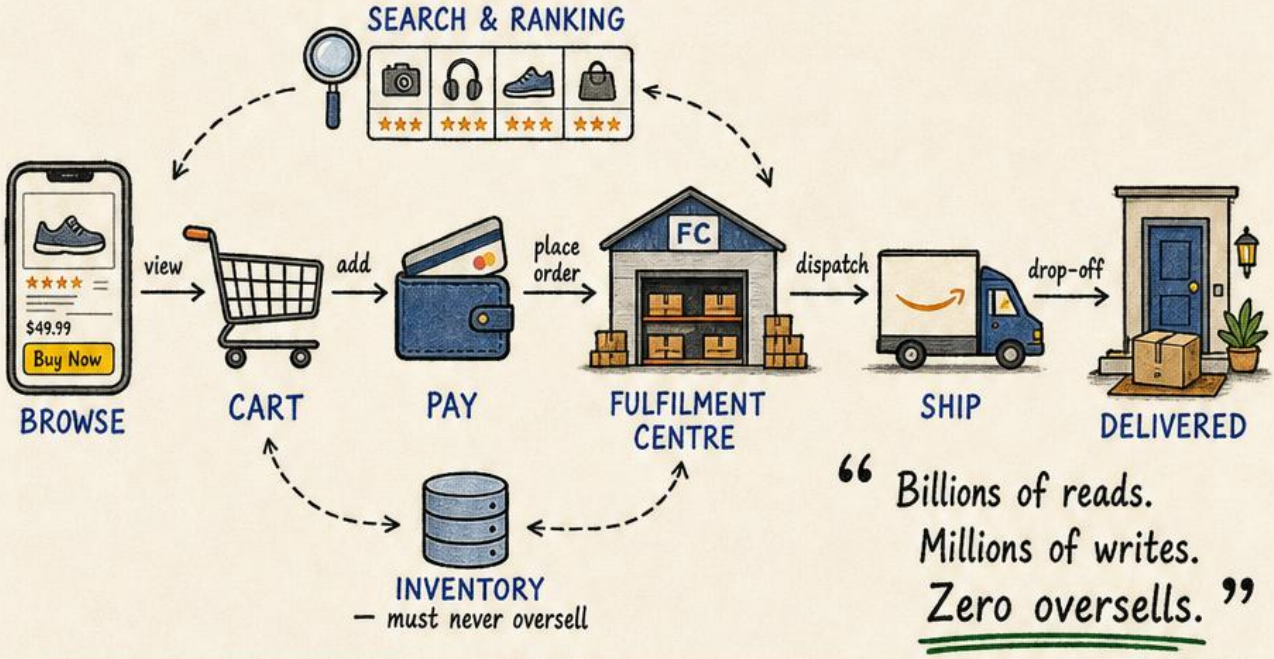
@ codedsoul_05

E-COMMERCE SD v1.0 ★

AMAZON SYSTEM DESIGN

— HANDWRITTEN HANDBOOK —

Designing a Planet-Scale E-Commerce & Fulfilment Platform



> Billions of reads.
> Millions of writes.
> Zero oversells.

Catalog • Search • Cart • Inventory • Orders • Fulfilment

What's Inside ?

1. Requirements & Scale
2. High-Level Architecture
3. Data Model & Storage
4. Product Catalog
5. Search & Ranking
6. Recommendations
7. Cart & Checkout
8. Inventory & Oversell
9. Payments & Orders
10. Fulfilment & Shipping

+ Flash Sales, Scaling & Interview Q&A

Amazon by the Numbers (Approx.)

- Customers — 300 Million+ active
- Products — 350 Million+ listings
- Orders — 1.6 Million/day peak
- Sellers — 2 Million+ third party
- Warehouses — 1,000+ worldwide
- Prime Day — 10× normal traffic

Read-heavy by default, write-critical at checkout.

Topics Covered

- ☑ Catalog Service
- ☑ Search & Relevance
- ☑ Recommendation Engine
- ☑ Cart Persistence
- ☑ Inventory Reservation
- ☑ Distributed Locking
- ☑ Order State Machine
- ☑ Payment Saga
- ☑ Warehouse Routing
- ☑ Delivery Tracking
- ☑ Flash Sale Hot Keys
- ☑ Caching & Sharding

Who is this for ?

- ★ System Design Interview Preparation
- ★ Backend / Full Stack Developers
- ★ Anyone building commerce at scale! ☺

> The catalog can be eventually consistent. The last item in stock cannot.

Browse Fast. Reserve Safely. Deliver Everywhere. ★

01 — REQUIREMENTS & SCOPE

Let anyone find any product in milliseconds — and guarantee that what they buy actually exists.

CORE FEATURES

- Browse and search a huge catalog
- Product detail pages with reviews
- Personalised recommendations
- Persistent cart and wishlist
- Checkout with multiple payment methods
- Order tracking to the doorstep
- Seller listings and inventory management
- Returns and refunds

THE THREE PARTIES



A marketplace: most inventory is not owned by the platform itself.

FUNCTIONAL REQUIREMENTS

1. Discovery

search, browse, recommend

2. Purchase

cart, checkout, payment

3. Inventory

reserve, decrement, restock

4. Fulfilment

pick, pack, ship, deliver

NON-FUNCTIONAL REQUIREMENTS

- Latency — product page under 200 ms
- Scale — hundreds of millions of listings
- Availability — the store must never be closed
- Consistency — strong for inventory and payment, eventual for catalog and reviews
- Read : write ratio ≈ 100 : 1
- Security — payments, addresses, seller data

OUT OF SCOPE

- ✗ AWS cloud infrastructure itself
- ✗ Prime Video / streaming
- ✗ Alexa and devices
- ✗ Advertising auction internals

INTERVIEW TIP

State the consistency split immediately: catalag, reviews and recommendations can lag; inventory, cart checkout and payments cannot. That one sentence frames the whole design.

Read-heavy everywhere, except the one row that matters.

02 — CAPACITY ESTIMATION

Browsing dwarfs buying — but buying is where correctness is expensive.

ASSUMPTIONS

Active customers	Daily active users	Orders per day	Product pages viewed per order	Catalog size	Peak multiplier (sale day)
= 300 Million	= 50 Million	= 1.5 Million	= 100	= 350 Million listings	= 10×

Illustrative estimates

READ TRAFFIC

Page views = 1.5M × 100 = 150 Million/day

Average = 150M ÷ 86,400 ≈ 1,730 views/sec

Peak (3× daily average) ≈ 5,200 views/sec

Sale-day peak ≈ 50,000 views/sec

This is a CDN and cache problem, not a database problem.

WRITE TRAFFIC

Orders = 1.5M ÷ 86,400 ≈ 17 orders/sec

Peak ≈ 200 orders/sec

Sale burst ≈ 2,000 orders/sec

Cart updates ≈ 10× orders ≈ 2,000/sec

Low volume, high value — every one of these must be correct.

STORAGE — CATALOG

350M listings × 10 KB metadata ≈ 3.5 TB

Images: 350M × 5 images × 200 KB
≈ 350 TB → object storage + CDN

Search index ≈ 2–3 TB across a sharded cluster

STORAGE — ORDERS

Order record ≈ 3 KB →
1.5M × 3 KB ≈ 4.5 GB/day

With items, events and replicas
≈ 30 GB/day ≈ 11 TB/year

Orders must be retained for years — archive cold, keep recent hot.

BANDWIDTH

Product page ≈ 500 KB with images

5,200 views/sec × 500 KB
≈ 2.6 GB/sec at peak

Over 90% served from CDN edge — origin sees a fraction.

WHAT THESE NUMBERS TELL US

100:1 read:write → cache and replicate aggressively

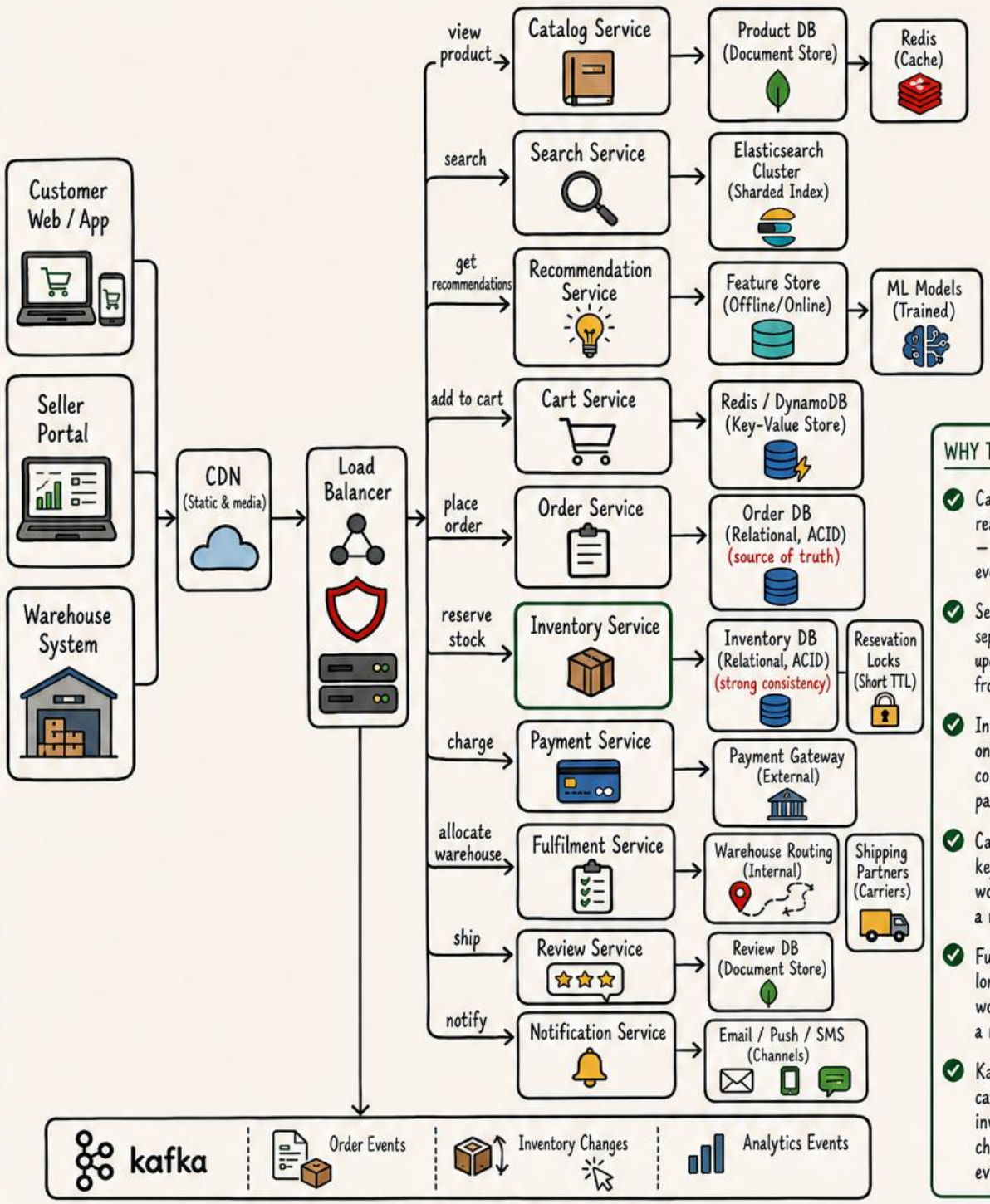
→ Images dominate bytes → CDN is mandatory

→ Order volume is small → afford strong consistency there

→ Sale days are 10× → pre-scale, queue, and shed load

Optimise reads for cost, writes for correctness. ★

03 — HIGH-LEVEL ARCHITECTURE



WHY THESE BLOCKS?

- Catalog is read-only at scale — cache it everywhere.
- Search is a separate index, updated asynchronously from the catalog.
- Inventory is the only strongly consistent hot path — isolate it.
- Cart is a key-value workload, not a relational one.
- Fulfilment is a long-running workflow, not a request.
- Kafka propagates catalog and inventory changes to every consumer.

PURCHASE FLOW

1. Search
2. View
3. Add to cart
4. Checkout
5. Reserve stock
6. Pay
7. Confirm order
8. Allocate warehouse
9. Pick & pack
10. Ship
11. Deliver

KEY DESIGN

- Microservices with clear ownership of data
- CQRS: write to the catalog store, read from caches and the search index
- Event-driven propagation for anything not on the checkout path

INTERVIEW TIP

Draw inventory as its own bordered box and say "this is where I need strong consistency". Interviewers wait for that.

Cache the browse path. Guard the buy path. ★

04 — APIs & CONTRACTS

REST for commerce actions. Events for everything downstream.

DISCOVERY APIs

- GET /v1/search?q=laptop&filters=&sort=&cursor=
- GET /v1/products/{product_id}
- GET /v1/products/{id}/offers — sellers, price, delivery date
- GET /v1/products/{id}/reviews?cursor=
- GET /v1/recommendations?user_id=&context=home

CART & CHECKOUT APIs

- POST /v1/cart/items — { product_id, seller_id, qty }
- PATCH /v1/cart/items/{id} — update quantity
- DELETE /v1/cart/items/{id}
- GET /v1/cart — with live price and availability
- POST /v1/checkout/session — locks price, reserves stock
- POST /v1/orders — Idempotency-Key required
- GET /v1/orders/{id} — status + tracking

INVENTORY APIs (internal)

- POST /internal/inventory/reserve — { sku, qty, ttl }
- POST /internal/inventory/commit — on payment success
- POST /internal/inventory/release — on failure or timeout
- GET /internal/inventory/{sku}/available

Reserve → commit → release.
Never decrement directly at add-to-cart.

SELLER APIs

- POST /v1/seller/listings
- PATCH /v1/seller/listings/{id}/price
- PATCH /v1/seller/inventory/{sku} — { quantity, warehouse_id }
- GET /v1/seller/orders

PLACE ORDER

```
{
  "idempotency_key": "ord_4f9a",
  "cart_id": "crt_881",
  "address_id": "adr_22",
  "payment_method": "card_tok_x91",
  "delivery_option": "standard"
}
```

IDEMPOTENCY



one key = one order, retries are safe

PRICE LOCK



the checkout session freezes price for N minutes

PAGINATION



cursor based; never OFFSET on 350M rows

RATE LIMITING



protect search and scraping endpoints



Re-check price and stock when the order is submitted.
A cart can sit open for days.

Reserve before you charge. Commit after you're paid. ★

05 — DATA MODEL & STORAGE

One product, many sellers, many warehouses — the joins are where the design lives.

USER
- user_id PK
- name
- email_hash
- addresses
- prime_flag

PRODUCT
- product_id PK
- title
- description
- category
- brand
- attributes
- images

SKU
- sku_id PK
- product_id
- seller_id
- variant attrs — size, colour

SELLER
- seller_id PK
- name
- rating
- fulfilled_by_platform_flag

OFFER
- sku_id PK
- seller_id CK
- price
- currency
- condition
- active

CART
- user_id PK
- items[]
- updated_at

ORDER
- order_id PK
- user_id
- status
- total
- placed_at
- address_snapshot

ORDER_ITEM
- order_id PK
- sku_id CK
- seller_id
- qty
- unit_price_snapshot

INVENTORY
- sku_id PK
- warehouse_id CK
- on_hand
- reserved
- available

SHIPMENT
- shipment_id PK
- order_id
- warehouse_id
- carrier
- tracking_no
- status

REVIEW
- product_id PK
- review_id CK
- user_id
- rating
- text
- verified_purchase

CRITICAL DESIGN DETAIL

PRODUCT is the catalog entity.
SKU is the sellable unit.
INVENTORY is per SKU PER WAREHOUSE.
available = on_hand − reserved.



One order can split into several shipments from several warehouses.

STORAGE CHOICES

- Document store — product catalog (nested, varied attributes per category)
- Relational (ACID) — orders, payments, inventory ledger
- Key-value store — cart, session, user preferences
- Elasticsearch — search index, rebuilt from catalog events
- Redis — hot product cache, price cache, stock counters
- Kafka — catalog updates, inventory changes, order events
- Object storage + CDN — images and media
- Warehouse / data lake — analytics, ranking and recommendation training

PARTITIONING

- Catalog sharded by product_id (hash) — uniform, no hot ranges
- Orders sharded by user_id — a user's history stays together
- Inventory sharded by sku_id — the reservation lock is local to one shard
- Reviews partitioned by product_id

Never shard inventory by warehouse — reservations would go cross-shard.

ACCESS PATTERNS (DRIVE THE DESIGN)

1. Product page by product_id — extremely hot
2. Offers and delivery estimate for one SKU
3. Search by keyword + filters
4. Cart by user_id
5. Order history by user_id, newest first
6. Available stock for one SKU across warehouses

Catalog is a document. Inventory is a counter. Orders are a ledger.
Three shapes, three stores.

Model the sellable unit, not just the product. ★

06 — PRODUCT CATALOG

A shoe and a laptop share almost no attributes — the catalog schema must bend without breaking.

— CATALOG STRUCTURE —

- **Category** (Electronics → Laptops)
  - **PRODUCT** (Model X Laptop) — shared title, description, images, reviews
  - **SKU / Variant** (16 GB, 512 GB, Silver)
    - OFFER by Seller A — ₹78,000, in stock, delivered by Fri
    - OFFER by Seller B — ₹79,500, in stock, delivered by Sun
    - OFFER by Seller C — ₹76,000, out of stock

Many sellers compete on the SAME product page. One of them wins the buy box.

THE BUY BOX

Which offer is shown by default?

Scored on: price + delivery speed + seller rating + fulfilment method + stock reliability.

Note: This is a ranking problem sitting on the most valuable pixel in commerce.

WHY A DOCUMENT STORE

- Attributes differ wildly per category (RAM vs shoe size vs expiry date)
- Read as one blob when the page loads
- Rarely joined at read time — denormalised on purpose

Note: Category-specific attribute schemas, validated at write time.

CATALOG WRITES ARE ASYNCHRONOUS

Seller updates a listing →

1. Validate and normalise
2. Write to the catalog store
3. Emit a catalog_changed event to Kafka
4. Search index, caches and recommendation features consume it

Note: Eventual consistency is fine here — a title change may take seconds to appear everywhere.

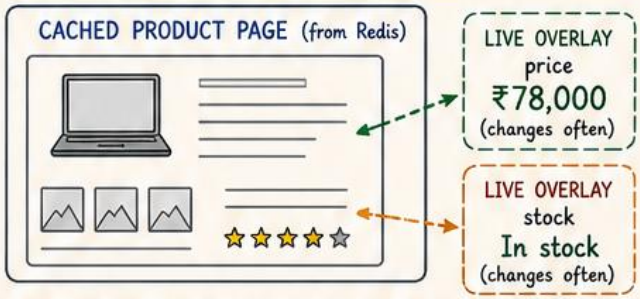
DATA QUALITY — THE REAL WORK

- Duplicate listings for the same physical product → deduplicate and merge
- Wrong or missing attributes → normalisation pipelines
- Counterfeit and prohibited items → moderation
- Image quality and standard sizes

Note: Catalog quality determines search quality. Garbage in, unfindable out.

SERVING THE PRODUCT PAGE FAST

- CDN for images and static fragments
- Redis cache of the assembled product document
- Price and stock fetched separately (they change often)
- Reviews loaded lazily below the fold



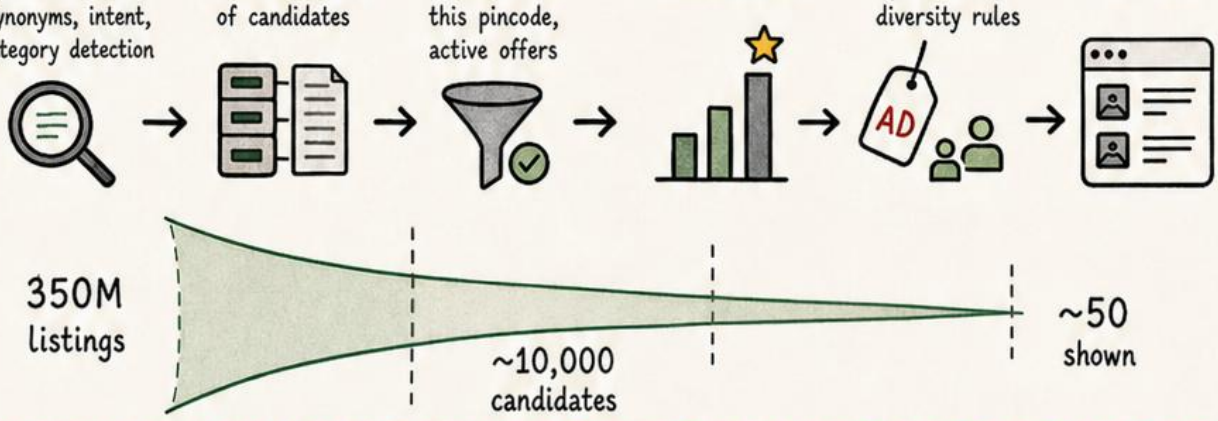
Cache the description for hours, the price for seconds.

07 — SEARCH & RANKING

Search is not a lookup. It is retrieval, then ranking, then business rules.

SEARCH PIPELINE

1. Query understanding — spell correction, synonyms, intent, category detection
2. Retrieval — inverted index returns thousands of candidates
3. Filtering — in stock, deliverable to this pincode, active offers
4. Ranking — machine-learned relevance score
5. Business layer — sponsored placements, diversity rules
6. Return a page with facets and cursors



INDEX DESIGN

- Inverted index: term → posting list of product ids
- Sharded by document; every shard searches, results are merged
- Replicas for read throughput and failover
- Near-real-time updates from Kafka catalog events

Note: Scatter-gather across shards, merge top-K at the coordinator.

RANKING SIGNALS

- Text relevance (BM25 / embeddings)
- Historical click-through and conversion rate
- Price competitiveness
- Delivery speed to this customer
- Seller rating and return rate
- Review count and average rating
- Personalisation from browse history

Conversion signals usually beat pure text relevance in commerce.

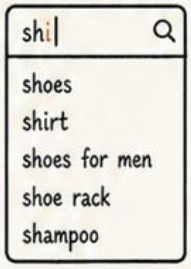
FACETS & FILTERS



Brand, price range, rating, delivery date, discount.

Computed as aggregations over the result set — expensive, so cache common combinations.

AUTOCOMPLETE



- Prefix trie or completion index in memory
- Ranked by query popularity + personal history
- Must respond in under 50 ms — served from a dedicated cluster.

HARD PROBLEMS

- ✗ Long-tail queries with no clicks to learn from
- ✗ Sellers keyword-stuffing titles
- ✗ Zero-result queries → relax filters and suggest alternatives
- ✗ Regional language and transliterated queries
- ✗ Keeping the index fresh during a price war

TIP

Say the two-stage split out loud:
cheap retrieval of many candidates,
then
expensive ranking of few.
That is how every large search system works.

Retrieve broadly, rank precisely, filter by reality.

08 — RECOMMENDATIONS

A large share of what people buy, they were not searching for.

ITEM-TO-ITEM COLLABORATIVE FILTERING

	ItemA	ItemB	ItemC	ItemD
User1	✓	✓		
User2	✓	✓	✓	
User3			✓	✓
User4	✓			✓

→ A ----- B

Customers who bought A also bought B (similarity 0.82)

Precompute item-to-item similarity offline. At request time it is just a lookup.

Items are far more stable than users — this is why item-based scales better.

PLACEMENTS (each is a different problem)

- Home feed — broad, personalised
- 'Frequently bought together' — complements (phone + case)
- 'Similar items' — substitutes (other phones)
- Post-purchase — accessories, not duplicates
- Cart page — small add-ons

Never recommend a substitute for something already purchased.

PIPELINE

OFFLINE (hours)	clickstream from Kafka →	feature store →	train models →	precompute candidates
NEAR REAL TIME (seconds)	session behaviour →	updates the user vector →	keep fresh	
ONLINE (imiliseconds)	fetch candidates →	filter →	re-rank →	render

CANDIDATE GENERATION → RANKING

1. Generate a few hundred candidates cheaply (co-purchase, category affinity, embeddings)
2. Then rank with a heavier model using price, availability and context.

COLD START

New user → popular in region, category bestsellers, session-only signals

New product → content-based similarity from attributes and text

Blend in some exploration or the catalog's tail never gets discovered.

FILTERS BEFORE SHOWING

- ✓ In stock and deliverable
- ✓ Not already owned
- ✓ Age-appropriate and policy-compliant
- ✓ Diverse — not five near-identical items

MEASURING

- Click-through rate
- Conversion
- Revenue per session
- Long-term retention

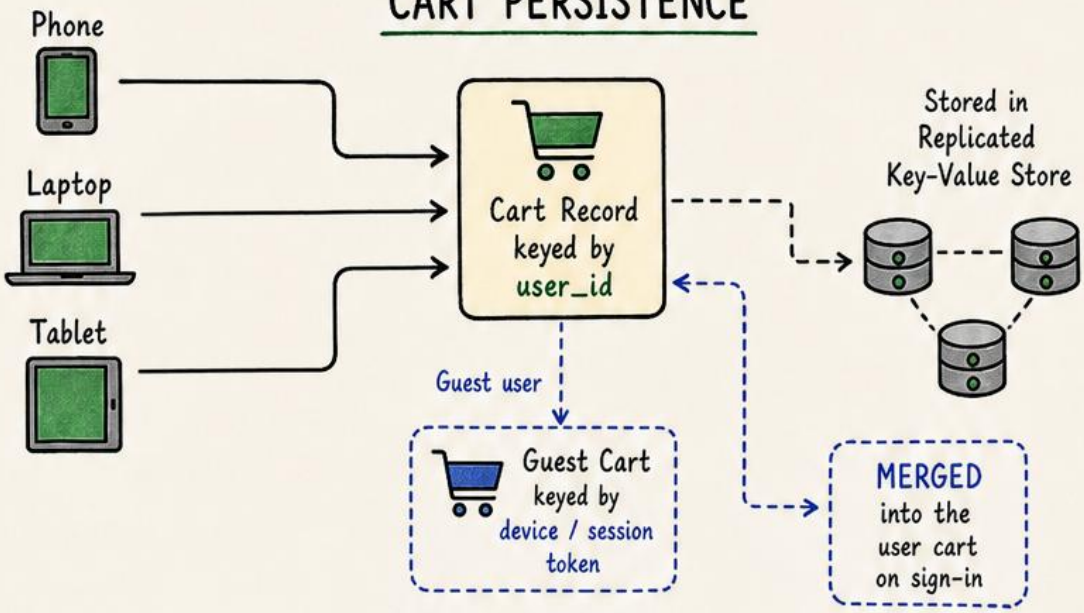
Optimise for the session, not the click — clickbait recommendations destroy trust.

★ Precompute the hard part offline, personalise in milliseconds. ★

09 — CART & SESSION

The cart must survive a closed tab, a dead battery and a switch from phone to laptop.

CART PERSISTENCE



Guest cart + user cart merge is the case candidates always forget.

WHY KEY-VALUE, NOT RELATIONAL

- Always accessed by a single key (user_id)
- No joins, no cross-user queries
- Very high write rate — every quantity tweak
- Needs to be always writable, even during partial outages

This is the classic shopping-cart use case for a highly available key-value store.

AVAILABILITY OVER CONSISTENCY

An unavailable cart loses a sale immediately.
Design choice: accept writes always, resolve conflicts later.

Conflict resolution when two devices edit at once:

- Merge the union of items
- Take the maximum quantity, or the latest timestamp per item

A resurrected deleted item is annoying.
A rejected add-to-cart is lost revenue.

WHAT THE CART DOES NOT DO

✕ Does **NOT** reserve inventory
✕ Does **NOT** lock the price

An item in your cart can sell out or change price — that is expected behaviour.

Note: Reservation happens at checkout, not at add-to-cart. Otherwise a few users could freeze the whole catalog.

CART CONTENTS

Store only references: sku_id, seller_id, quantity, added_at.

Fetch live price, availability and delivery estimate on every read.

Note: Never store the price in the cart — display it fresh.

SAVE FOR LATER & WISHLIST

Different lifecycle from the cart: long-lived, no checkout intent, good place for price-drop alerts.

ABANDONED CARTS

Carts persist for months.
Emit an event after N hours of inactivity → reminder notification, price-drop alert.

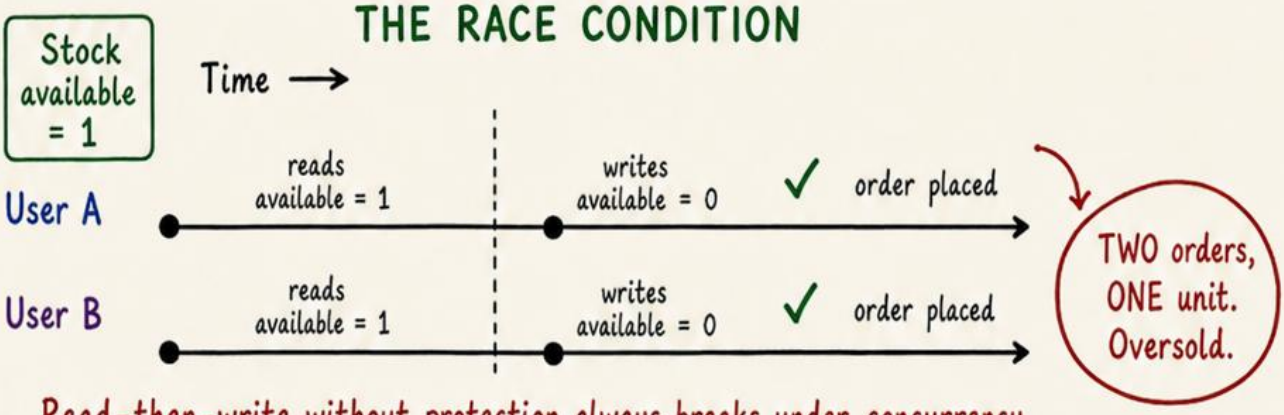
Note: Cart abandonment recovery is one of the highest-return features in commerce.

Always writable, never authoritative.

10 — INVENTORY & THE OVERSELL PROBLEM

Everything else can be eventually consistent. The last item in stock cannot.

THE RACE CONDITION



Read-then-write without protection always breaks under concurrency.

THE THREE-STATE MODEL

- on_hand — physically in the warehouse
- reserved — promised to in-flight checkouts
- available = on_hand − reserved



RESERVE → COMMIT → RELEASE

1. RESERVE at checkout start, with a TTL (e.g. 10 minutes)
2. COMMIT on payment success → on_hand decreases, reserved decreases
3. RELEASE on failure, cancellation or TTL expiry → reserved decreases

Note: The TTL is what stops abandoned checkouts from freezing stock forever.

ATOMIC CONDITIONAL UPDATE

```
UPDATE inventory
   SET reserved = reserved + 2
 WHERE sku_id = 'SKU_91'
   AND warehouse_id = 'WH_3'
   AND on_hand - reserved >= 2;
-- rows affected = 0 → not enough stock, reject
```

Note: Let the database do the checking. One statement, no read-then-write gap.

OTHER APPROACHES

- Pessimistic lock — SELECT ... FOR UPDATE. Correct, but serialises a hot row.
- Optimistic lock — version column, retry on conflict. Good when contention is low.
- Distributed lock (Redis) — works, but the database is still the final arbiter.
- Atomic counter in Redis — very fast, needs careful durability and reconciliation.

Note: For flash sales, a Redis counter in front with async database reconciliation is common.

MULTI-WAREHOUSE

Stock lives in many warehouses. Reserving means choosing WHERE to reserve from: nearest warehouse with stock → fastest delivery promise.
If no single warehouse has everything, split the order into shipments.

Note: Availability shown to the customer is aggregated; reservation is always per warehouse.

WHY NOT RESERVE AT ADD-TO-CART?

Millions of carts would hold stock hostage.

Reserve only at checkout, for a short window.

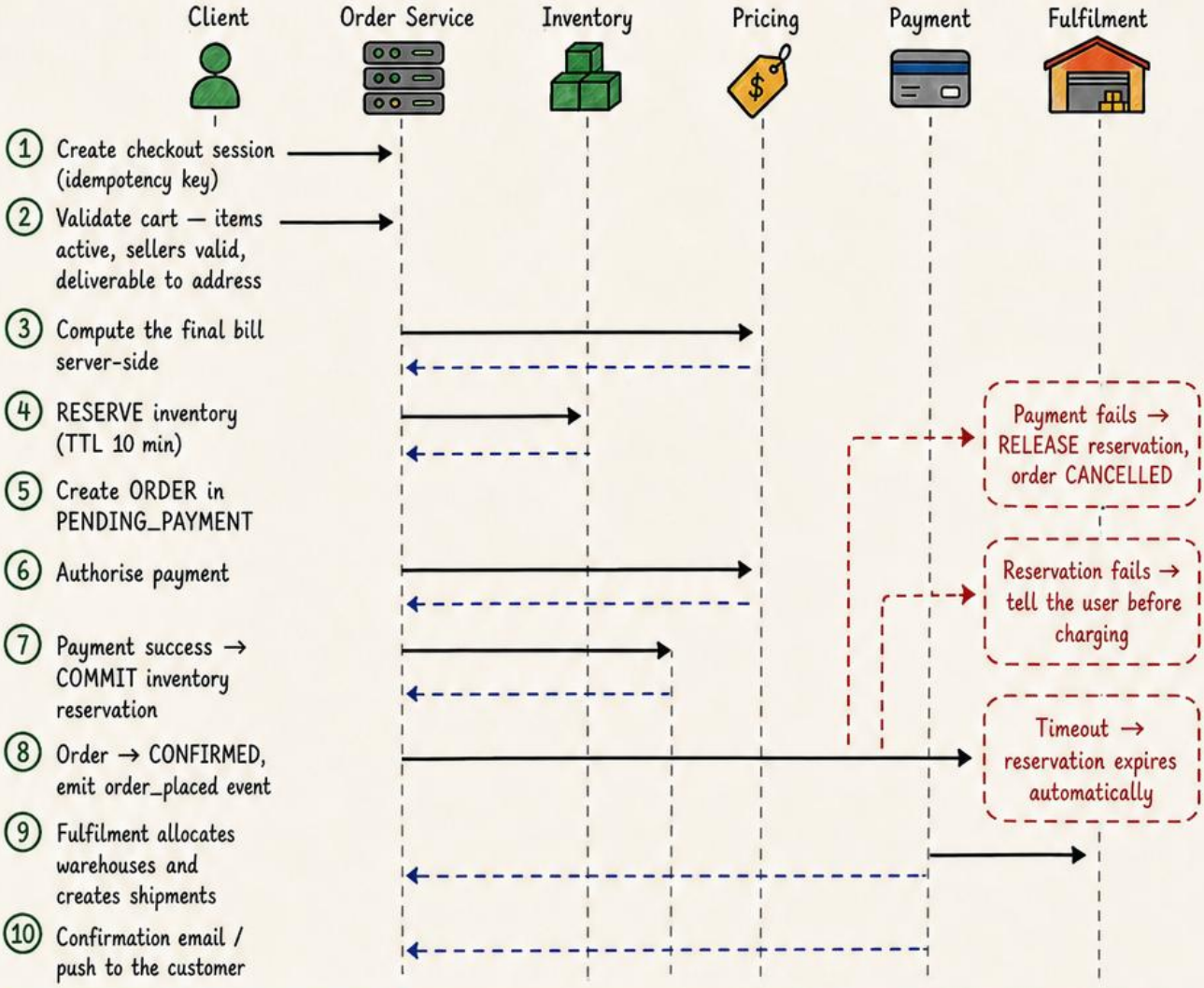
RECONCILIATION

Physical counts drift: damage, theft, misplaced items, returns.
Run periodic cycle counts and correct the ledger; keep an append-only inventory event log for audit.

Make the database enforce the invariant.
Never the application. ★

11 — CHECKOUT & ORDER PLACEMENT

Checkout is a distributed transaction across services that cannot share a database.



ORDER OF OPERATIONS MATTERS

- Reserve stock BEFORE charging.
- Charging first and then discovering there is no stock means a refund, an apology and a lost customer.

Note in green: Fail on the cheap step first.

SAGA, NOT TWO-PHASE COMMIT

- Services own their own data; no global transaction.
- Each step has a compensating action:
  reserve ↔ release • authorise ↔ void
  confirm ↔ cancel + refund
- An orchestrator drives the saga and retries with idempotency keys.

forward steps

compensating actions

BILL CALCULATION (server-side)

Item subtotal
+ Shipping charges
+ Taxes (destination-based)
− Coupons and promotions
− Gift card balance

= Order total

Never trust a total from the client. Recompute and compare.

SPLITTING THE ORDER

One order may become several shipments:

- different sellers
- different warehouses
- different delivery speeds
- oversized items.

Each shipment tracks independently, but the customer sees one order.

IDEMPOTENCY

- Same key → same order returned.
- Protects against double-click, retry storms and flaky mobile networks.
- Store key → order_id with a TTL.

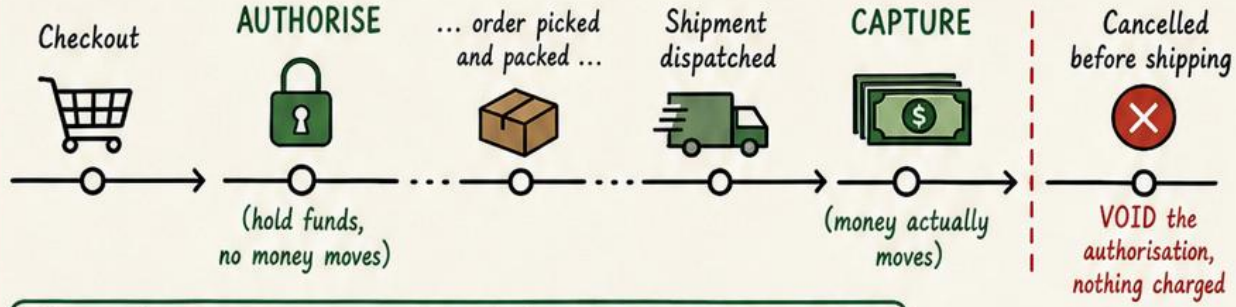
Draw the compensating arrows. Saying "saga with compensation" without showing the rollback path convinces nobody.

Reserve, then charge, then commit — and be able to undo each one.

12 — PAYMENTS & ORDER STATE

In commerce you authorise at checkout and capture when the box ships.

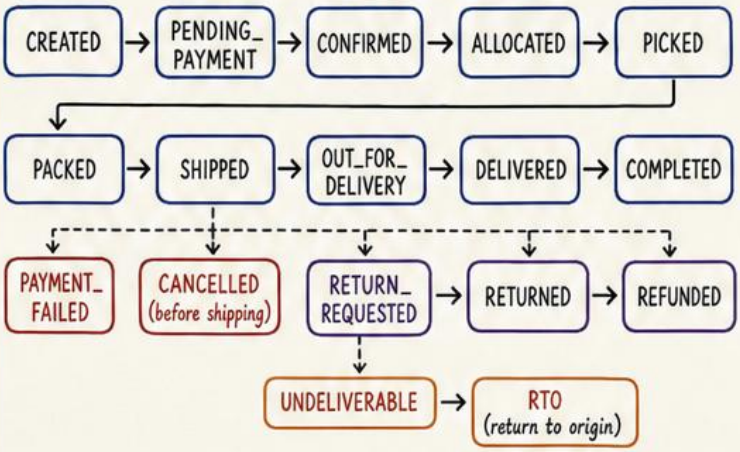
AUTHORISE vs CAPTURE



This is why an e-commerce cancellation before dispatch is instant and free.

Authorisations expire (often ~7 days). Long-delayed shipments need re-authorisation.

ORDER STATE MACHINE



Note: Every transition is an append-only event, not just a column update.

PAYMENT METHODS

- Cards (tokenised)
- UPI / bank transfer
- Wallets
- Net banking
- EMI
- Gift cards and store credit
- Cash on delivery
- BNPL — Buy now pay later

Note: Split tender — part gift card, part card — must be handled atomically.

REFUNDS & RETURNS

- Cancelled before dispatch → void the authorisation
- After delivery → return window → item received and inspected → refund issued
- Partial order returned → partial refund, recompute taxes and discounts

Note: Refunding a discounted multi-item order correctly is genuinely hard — model discount allocation per item.

THE LEDGER

Append-only, double-entry style records:



- Immutable
- Reconciled daily against gateway statements
- Source of truth for finance, not the order table

SAFEGUARDS

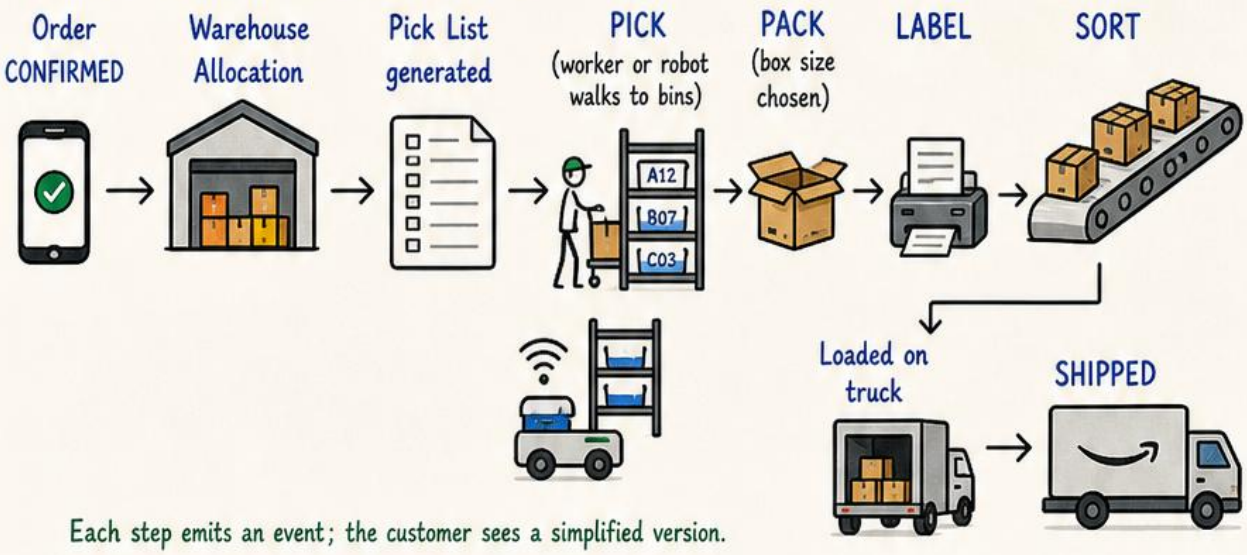
- Idempotency keys on every money operation
- Webhooks → polling reconciliation (never trust the client callback)
- Retries with exponential backoff, dead-letter queue for stuck items
- PCI compliance — tokenise, never store raw card numbers
- Fraud scoring before high-value authorisation
- Alerting on refund spikes and settlement mismatches

Hold the money at checkout. Take it when the box leaves.

13 — FULFILMENT & WAREHOUSES

The order is now a physical object that must be found on a shelf.

INSIDE THE FULFILMENT CENTRE



Each step emits an event; the customer sees a simplified version.

WAREHOUSE ALLOCATION — THE DECISION

Which warehouse ships this order?

Score on:

- Has all the items in stock
- Distance to the delivery address
- Current warehouse workload and cut-off time
- Shipping cost
- Whether splitting into two shipments beats one slow shipment

Fewest shipments AND fastest delivery are in tension — pick a policy and state it.

PICKING OPTIMISATION

A naive pick walks kilometres per order.

- Batch picking — collect items for many orders in one route
- Zone picking — workers own a zone, items merge later
- Route optimisation through the aisles
- Chaotic (random) storage — popular items placed near packing

This is a travelling-salesman-shaped problem solved heuristically, thousands of times an hour.

INVENTORY PLACEMENT

Deciding WHERE to stock a product before anyone orders it.

Driven by demand forecasting per region.

Good placement is why two-day delivery is possible at all — the item was already near you.

SHIPMENT CREATION

One order → N shipments.

Each shipment: warehouse, carrier, tracking number, expected delivery window.

Customer sees a merged view; internally they are independent workflows.



EXCEPTIONS

- Item not found on the shelf (phantom inventory) → reallocate to another warehouse
- Damaged during packing → repick
- Missed carrier cut-off → next-day dispatch, update the promise
- Address undeliverable → hold, contact customer, or return to origin

THE DELIVERY PROMISE

Shown BEFORE the order is placed, so it must be computed fast:



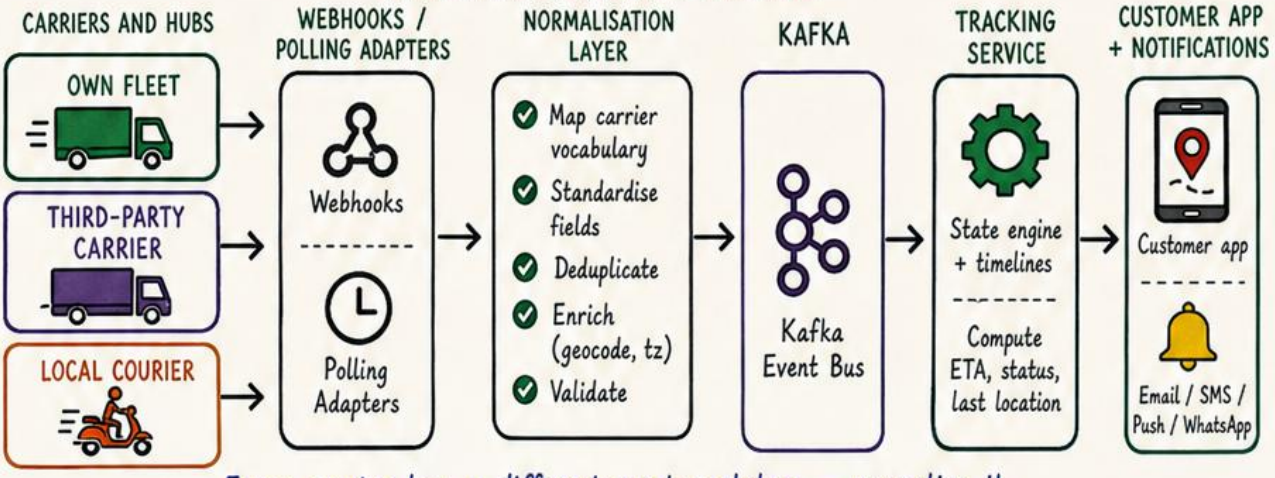
Promise conservatively; measure on-time delivery rate obsessively.

Software ends at the shelf, but the promise does not.

14 — SHIPPING & TRACKING

Once the box leaves, your system becomes an aggregator of other people's events.

TRACKING EVENT PIPELINE



Every carrier has a different event vocabulary — normalise it.

THE JOURNEY (events)

Shipped from warehouse	10:12 AM, BLR
Arrived at sorting hub	02:43 PM, BLR
Departed hub	04:10 PM, BLR
Arrived at local delivery station	07:22 AM, HYD
Out for delivery	10:05 AM, HYD
Delivery attempted	01:41 PM, HYD
Delivered	02:32 PM, HYD

Show the customer a clean timeline; hide the twelve internal scans behind it.

NORMALISATION IS THE REAL WORK

- CARRIER A says "IN_TRANSIT"
- CARRIER B says "Scan: Hub Out"
- CARRIER C sends a CSV at midnight

→ Map all of them into one internal status vocabulary.

Out-of-order and duplicate events are normal — dedupe by event id, order by timestamp, never regress the status.

LAST-MILE DELIVERY



Full live GPS tracking, like ride hailing.



Only discrete scan events, no live map.

Design both paths — the UI must degrade gracefully when live location is unavailable.

DELIVERY CONFIRMATION

- OTP for high-value items
- Photo proof at the doorstep
- Signature for restricted goods
- Geofence check against the delivery address

Proof of delivery is what makes "I never got it" disputes resolvable.

ETA UPDATES

Recompute the promised date when:

- Shipment dispatched
- Hub scan is delayed
- Weather or strike disruption
- Delivery attempt fails

Push proactive updates — silence during a delay is what generates support tickets.

FAILED DELIVERY

1. Attempt 1 fails
2. Reattempt schedule
3. Customer contacted
4. Held at a pickup point
5. After N attempts, return to origin and refund.

RTO (return to origin) is expensive — reducing it is a core business metric.

NOTIFICATIONS



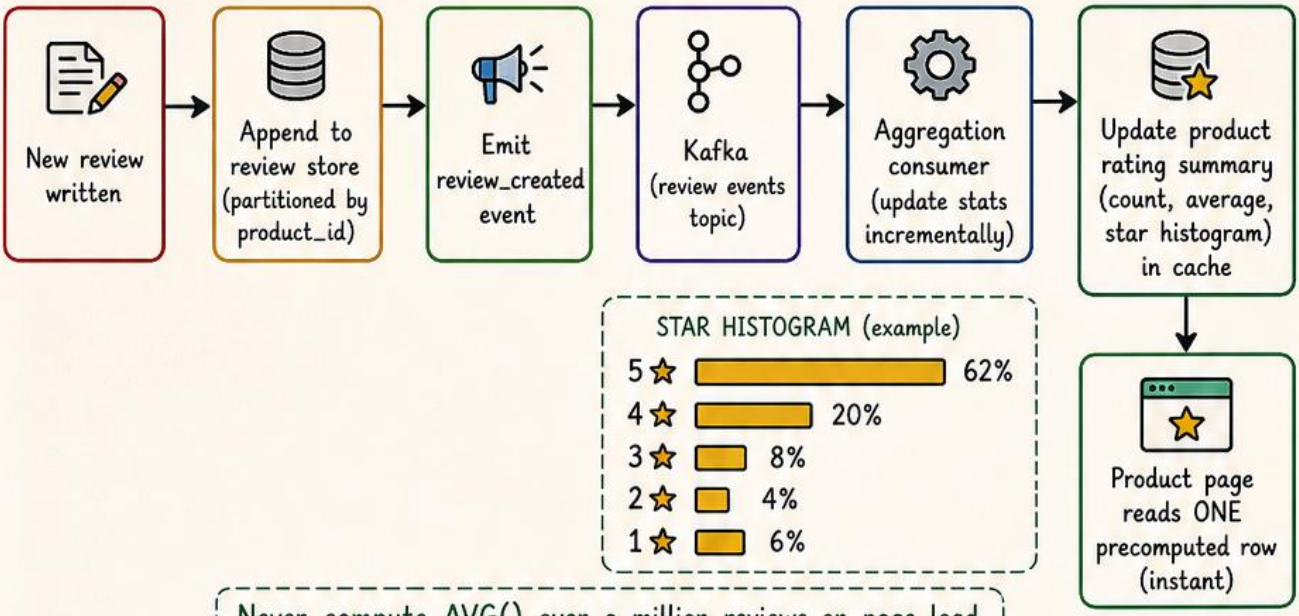
Batch and rate-limit them; one message per meaningful change, not per internal scan.

★ Aggregate messy events into one honest timeline. ★

15 — REVIEWS, RATINGS & TRUST

Reviews are written rarely and read constantly — the perfect precomputation problem.

— RATING AGGREGATION —



Never compute AVG() over a million reviews on page load.

Note: Incremental update: new_avg = (old_avg × n + rating) ÷ (n + 1).

WRITE PATH

1. Verify the user actually purchased the item
2. Moderate — profanity, spam, policy, images
3. Store the review (partitioned by product_id)
4. Emit an event for aggregation and search boosting

Note: Verified-purchase badges are the cheapest, strongest trust signal.

READ PATH

- Summary (stars + count) — cached hard, shown instantly
- Review list — paginated, cursor-based, sorted by helpfulness
- 'Most helpful' ranking from helpful-votes and recency

Load reviews lazily — they are below the fold.

RANKING REVIEWS

- Helpfulness votes
- Verified purchase
- Length and detail
- Recency (a 5-year-old review of a phone is noise)
- Reviewer credibility

Note: Show a representative mix, not just the most positive.

FIGHTING FAKE REVIEWS

- ✗ Review farms and paid reviews
- ✗ Sudden bursts of 5★ from new accounts
- ✗ Review hijacking — a listing changes product after collecting reviews
- ✗ Competitor sabotage with 1★ floods

Defences: purchase verification, velocity anomaly detection, account graph analysis, device fingerprints, manual review of flagged spikes.

CONSISTENCY

Eventual is fine. A review appearing 30 seconds later harms nobody.

Note: Show the author their own review immediately (read-your-writes) even while it propagates.

SCALE NOTE

Reviews are append-heavy and read-enormous.

Partition by product_id, cache the summary, and keep the full text in a store optimised for large blobs.

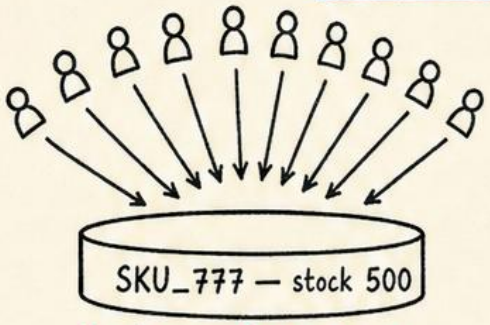
★ Write once, moderate carefully, read a million times from cache. ★

16 — FLASH SALES & HOT KEYS

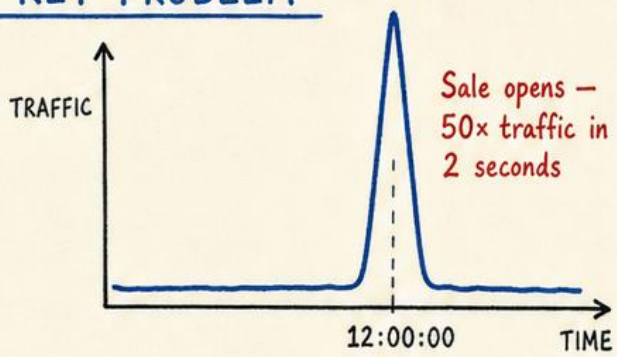
★ KEY PAGE

A million people pressing buy on the same row at the same second breaks every normal assumption.

THE HOT KEY PROBLEM



Single row. Single shard. Serialised writes.



Sharding does not help.
All the load lands on one key by definition.

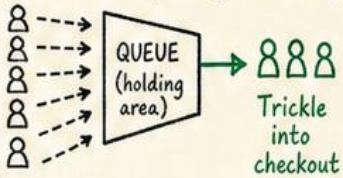
DEFENCES — BEFORE THE SALE

- Pre-warm caches with the deal products
- Pre-scale infrastructure on a schedule, not reactively
- Static-render the deal page and push it to the CDN
- Load-test with the real expected spike shape

Autoscaling reacts in minutes. The spike lasts seconds. Pre-scale.

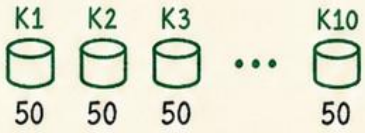
DEFENCES — DURING THE SALE

- Waiting room / virtual queue — admit users in controlled batches
- Redis atomic counter (DECR) as the first gate; database reconciles asynchronously
- Request coalescing — one cache miss fetches for all waiters
- Per-user purchase limits enforced early
- Fail fast with a clear 'sold out' rather than queueing doomed requests



COUNTER SHARDING

Split stock of 500 into 10 buckets of 50 across 10 keys.



Each request hits a random bucket → 10× the write throughput.

Trade-off: a bucket can be empty while others have stock — rebalance or retry another bucket.

Classic technique. Mention it and explain the trade-off.

PRICING & DEALS ENGINE

Deal types: lightning deals (time + quantity limited), coupons, bulk discounts, subscription pricing.

Price changes propagate through the same event pipeline as the catalog — but with a much shorter cache TTL.

Price and stock must never be served from a long-TTL cache.

FAIRNESS & ABUSE

Bots buy faster than humans.

- Rate limit per account, device and IP
- CAPTCHA or proof-of-work on the deal endpoint
- Account age and reputation gates
- Detect and cancel bulk scalper orders

Fairness is a product requirement, not just an engineering one.

GRACEFUL BEHAVIOUR

If overloaded:

- Degrade recommendations
- Disable non-essential widgets
- Keep search and checkout alive

★ You cannot shard a single hot row — so queue, cache and count in memory. ★

17 — SCALING & CACHING

At a 100:1 read-to-write ratio, caching is not an optimisation — it is the architecture.

CACHE HIERARCHY

1. Browser cache — static assets → ~80–90% hit rate
2. CDN edge — images, static page fragments (~60% of requests) → ~60% hit rate
3. API / edge cache — assembled product documents → ~40% hit rate
4. Redis — hot products, prices, session, cart → ~35% hit rate
5. Application local cache — configs, category trees → ~10% hit rate
6. Database — only ~1–5% of reads reach here → ~1–5% hit rate

Each layer absorbs an order of magnitude.

WHAT TO CACHE FOR HOW LONG

- Images and description — hours to days
- Product document — minutes
- Search results for popular queries — minutes
- Price — seconds
- Stock availability — seconds, or never (read through)

The right TTL differs per field. One cache entry for the whole page is a mistake.

SHARDING STRATEGY

- Catalog → hash by product_id (uniform distribution)
- Orders → by user_id (history stays together)
- Inventory → by sku_id (reservation stays on one shard)
- Reviews → by product_id
- Search → document-sharded index with replicas

Choose the shard key from the access pattern, not from the entity name.

READ REPLICAS & CQRS

Writes → primary. Reads → replicas and derived views.

Search index, recommendation features and analytics are all read models built from the event stream.

Accept replication lag everywhere except inventory and payment.

ASYNC EVERYTHING NON-CRITICAL

KAFKA

Kafka carries:

- catalog updates
- inventory changes
- order events
- clickstream
- emails
- invoices
- analytics
- seller notifications

Synchronous only:

- ✓ product read
- ✓ cart write
- ✓ reserve stock
- ✓ authorise payment
- ✓ create order

GLOBAL DISTRIBUTION

Region-local catalog replicas and CDNs.

Orders and inventory pinned to the region that owns the warehouse.

Serve reads from anywhere; write where the data lives.

DATABASE HYGIENE

- Connection pooling
- Short transactions
- No cross-shard joins
- Archive old orders to cold storage
- Avoid OFFSET pagination on huge tables.

★ Push reads outward, keep writes narrow and short. ★

18 — FAILURES & RELIABILITY

Every second the buy button is broken is measurable revenue lost.

WHAT BREAKS	→ WHAT WE DO
Search cluster down	serve category browse + cached popular results
Recommendation service down	fall back to bestsellers in category
Product cache miss storm	request coalescing, serve stale while revalidating
Inventory service down	block checkout, keep browsing alive (never guess stock)
Payment gateway down	failover to a secondary provider; queue and retry
Order database primary fails	automatic replica promotion, brief write pause
Warehouse system offline	accept orders, allocate later from a queue
Carrier webhook silent	poll their API, show last known status with a timestamp
Kafka consumer lag	prioritise order events, delay analytics
Region outage	shift traffic to another region; catalog reads unaffected

GRACEFUL DEGRADATION LADDER



Note: Checkout is the LAST thing to switch off — and if inventory is uncertain, it must be switched off rather than oversell.

PROTECTING THE HOT PATH

- Circuit breakers on every downstream call
- Timeouts with sensible defaults everywhere
- Retries with exponential backoff and jitter, only for idempotent operations
- Bulkheads — a failing review service must not slow the product page
- Load shedding by request priority (checkout > browse > analytics)

DATA SAFETY

- Multi-region replication for orders and payments
- Point-in-time recovery and tested restores
- Append-only event log so state can be rebuilt
- Reconciliation jobs for inventory and money

Note: A backup you have never restored is a hypothesis, not a backup.

WHAT TO MONITOR

- Orders per minute vs forecast
- Checkout success rate • Add-to-cart rate
- p99 product page latency
- Cache hit ratio • Inventory reservation failures
- Payment authorisation success
- Oversell incidents (must be zero)

Note: Alert on a drop in orders per minute — it detects outages your health checks miss.

SAFE DEPLOYMENT

Canary → 1% → 10% → full

with automatic rollback on error-rate or conversion regression.
Feature flags for anything touching checkout.

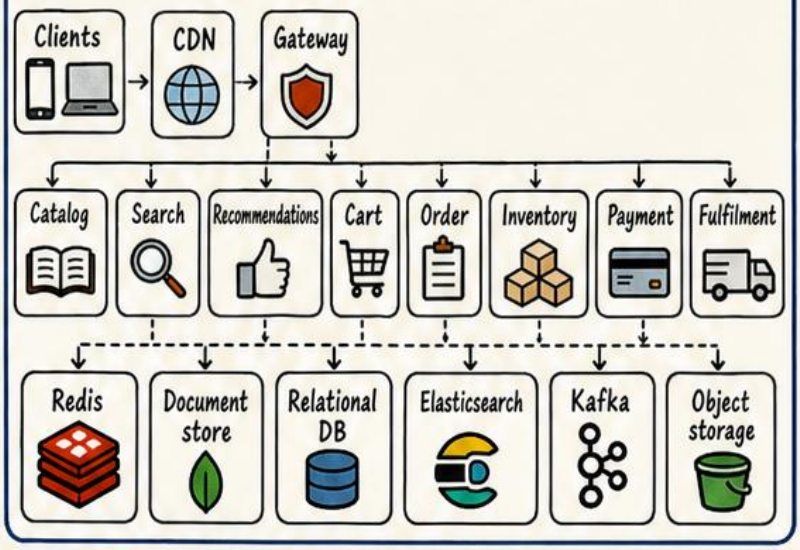
★ Fail soft on browse. Fail closed on inventory. ★

19 — RECAP & INTERVIEW CHEAT SHEET

THE WHOLE SYSTEM IN ONE LINE

Serve a giant catalog from cache, rank it well, reserve stock atomically before taking money, then turn the order into boxes moving through warehouses.

MINI ARCHITECTURE RECAP



TOP 10 TALKING POINTS

1. Split consistency: eventual for catalog, strong for inventory and money
2. 100:1 read:write → CDN + multi-layer cache is the architecture
3. Product vs SKU vs Offer vs Inventory-per-warehouse
4. Search = cheap retrieval then expensive ranking
5. Cart is highly available key-value, never authoritative
6. Reserve → commit → release with a TTL prevents oversell
7. Atomic conditional UPDATE, not read-then-write
8. Checkout is a saga with compensating actions
9. Authorise at checkout, capture at shipment
10. Flash sales are a hot-key problem: queue, count in Redis, shard the counter

NUMBERS TO REMEMBER

- 350M listings
- 1.5M orders/day
- ~1,730 page views/sec average
- 50,000 views/sec on sale day
- 100:1 read:write
- 10 min reservation TTL
- under 200 ms product page

MISTAKES TO AVOID

- X Read-then-write on inventory
- X Charging before reserving stock
- X Caching price and stock with a long TTL
- X Treating one order as one shipment
- X Ignoring the guest-to-user cart merge

QUICK Q & A

Q: How do you prevent overselling?
A: Reserve with a TTL and use one atomic conditional update guarded by on_hand − reserved >= qty.

Q: Why not reserve stock at add-to-cart?
A: Millions of idle carts would hold inventory hostage.

Q: Why a saga instead of a distributed transaction?
A: Services own separate databases; two-phase commit does not scale and blocks on failure.

Q: How do you handle Prime Day?
A: Pre-scale, pre-warm caches, virtual queue, Redis counters, shed non-essential features.

Q: Why authorise then capture?
A: Cancellation before dispatch costs nothing, and funds are guaranteed at checkout.

Q: Can search be eventually consistent?
A: Yes — the index is rebuilt from catalog events; stock is filtered at read time.

CACHE EVERYTHING. RESERVE ATOMICALLY. NEVER OVERSELL.